

Orbital — Project Overview

Hack-a-Claw x NVIDIA @ UCSC | May 15–16, 2026 Track: Edge (ASUS Ascent GX10) + NemoClaw Bonus

1. What We Are Building

Orbital is an on-device autonomous agent that does conjunction triage and maneuver planning for satellites in low Earth orbit. It runs entirely on the ASUS Ascent GX10, with no cloud dependency.

In one sentence: **Orbital watches the orbital catalog, detects close approaches, reasons about which ones matter, and drafts maneuver recommendations for a human flight director to approve.**

The agent does the analyst's job. Humans approve. Spacecraft execute. This is how NASA CARA, ESA, and commercial operators already run conjunction operations — Orbital is not replacing the chain of command, it is automating the highest-effort cognitive layer inside it.

2. Why This Matters

Low Earth orbit has over 22,000 tracked objects, with 200 to 400 close approaches detected per day. SpaceX alone executes roughly 100,000 collision-avoidance maneuvers per year and the number is climbing. Most operators get conjunction warnings (CDMs) that they triage by hand or with expensive commercial tools.

The actual problem is not detection — the math for that is solved. The problem is **triage**. Of the hundreds of warnings issued per day, most are statistical noise. The hard, expertise-heavy work is deciding which ones warrant a maneuver burn and how to plan that maneuver against fuel budgets and other upcoming conjunctions.

Orbital targets the gap: a deployable, edge-resident agent that does this triage on-device, suitable for university smallsat programs, smallsat startups, and air-gapped defense operators who cannot send sensitive orbital data to cloud LLMs.

3. The Project in One Picture

The system has three layers, with humans in the loop at the top.

Layer 1: Orbital Engine. Pulls TLE data from Celestrak, propagates a curated catalog of ~500 to 1000 satellites using SGP4, and screens all pairs for close approaches over a 24 to 72 hour horizon. Pushes flagged events into a queue. This layer is deterministic math, runs continuously, and does no reasoning.

Layer 2: Agent. Pulls events from the queue, runs a multi-step reasoning loop on each, decides whether the event is noise, watch-only, or action-required. For action-required events, generates and evaluates multiple maneuver plans and drafts a recommendation. Powered by Nemotron, orchestrated via OpenClaw.

Layer 3: UI and Human Approval. Two views. Operator View shows the live 3D Earth, flagged events, and agent reasoning stream. Approver View shows pending maneuver recommendations with approve/reject controls. When a human approves a plan, the maneuver simulator applies it to the running orbital state and the visualization shows the conjunctions resolving.

4. The Agent

Operating modes. The agent runs in one of three modes. **Idle/Monitor** is the default state — every 30 seconds it emits a heartbeat status check ("Monitoring 847 objects, no new flags in the last interval"). **Investigate** is triggered when a new conjunction is flagged — the agent runs a multi-step tool-use loop to refine the analysis. **Plan** is triggered when investigation concludes a maneuver is warranted — the agent gathers all upcoming conjunctions for the asset and generates candidate plans.

Pc thresholds. Probability of collision drives the decisions. Below $1e-6$: dismiss as noise, log only. Between $1e-6$ and $1e-4$: watch, re-screen in 6 hours. At or above $1e-4$: evaluate maneuvers. (Industry standard action threshold is 1 in 10,000.)

The reasoning loop. When a conjunction is flagged, the agent acknowledges the event, fetches metadata on both objects, queries memory for prior events involving the asset, re-propagates with fresh TLEs to rule out stale data, checks current space weather and inflates covariance accordingly, computes a refined P_c , and decides which mode to enter. For action-required events, it pulls all other upcoming conjunctions for the asset, generates two candidate plans (typically a single-burn approach and a split-burn approach), evaluates each for total Δv and conjunctions resolved, and drafts a recommendation with one alternative.

The tool surface. The agent has functions for fetching flagged conjunctions, object metadata, space weather, asset-level conjunctions, and memory; for analysis (re-propagation, Pc computation, maneuver simulation, plan evaluation); and for output (draft recommendation, stream thought). Stream-thought is called constantly so the operator and judges can watch the agent reason in real time.

Memory. A single SQLite table logs every event: detected, dismissed, recommended, approved, rejected, executed. Includes asset ID, conjunction IDs, Pc, miss distance, verdict, approver, and context (space weather snapshot, etc.). Indexed for fast lookups. The agent queries memory before every recommendation to surface prior decisions on similar events.

5. Datasets

Required for the demo to work. Celestrak TLEs for the curated satellite catalog (free, public, no auth) — use the Starlink shell, ISS, and Fengyun-1C debris groups specifically. NOAA SWPC space weather feeds (free, public, JSON) — Kp index, X-ray flux, geomagnetic storm level.

Strongly recommended. Space-Track.org real CDMs if registration approval comes through — apply now since approval can take days. SatCat metadata for human-readable object info.

Visual polish. NASA Visible Earth textures for the 3D sphere. ISS live position from Open Notify for the demo's hero shot.

Scope decision. We are running ~500 to 1000 objects, not the full 22,000-object catalog. The chosen subset is the 500–600km Starlink shell plus ISS plus the 2007 Fengyun-1C debris cluster at 800km. This gives us real, named, dramatic conjunctions instead of "object 47291 vs object 12044."

6. The UI

Operator View is the main dashboard. Top KPI strip shows critical conjunctions, satellites under watch, resolved events, total tracked, and current space weather (Kp index). Left panel lists currently flagged conjunctions sorted by Pc, descending. Center is the 3D Earth with the satellite catalog, conjunctions flashing red when detected. Right panel is the Agent Activity stream — live, scrolling, token-by-token reasoning output from the agent. Bottom strip is the 72-hour timeline showing upcoming flagged events.

Clicking a satellite slides up a drawer with detail on that asset: Pc history chart, past conjunction events, and the agent's current verdict if one is in progress.

Approver View is a focused queue of pending maneuver recommendations. Each recommendation appears as a card showing the asset, the conjunctions it resolves, the proposed burn sequence with Δv and timing, total fuel cost, the agent's reasoning paragraph, and an alternative plan. Approve and Reject buttons. When the operator approves, the maneuver simulator applies the plan and the Operator View updates — the conjunctions resolve, the satellite's orbit subtly shifts, the marker turns green.

A role-switcher in the top-right lets the demo presenter toggle between Operator and Approver views.

7. The NemoClaw Story

NemoClaw wraps the agent in a hardened sandbox with policy-gated egress. For Orbital, this matters because orbital state data and CDMs are operationally sensitive — defense and commercial operators cannot send them to cloud LLMs. With NemoClaw:

- Raw orbital state and CDMs stay inside the sandbox
- Only sanitized outputs (the maneuver recommendation, the approver's decision) cross the policy boundary
- The agent reasons about classified or commercial data locally with no risk of leakage

If NemoClaw is too rough in early preview to make the build work, we drop it without affecting the main project. It is the bonus track, not the foundation.

8. Team Roles

We are four people. Each owns one of four workstreams. Hand-offs happen through pre-defined data contracts.

Orbital Engine Lead. Owns TLE parsing, SGP4 propagation, conjunction screening, Pc computation (Foster method), maneuver simulation, plan evaluation. Pure math and data plumbing. Day 0 deliverable: parse a Celestrak TLE and propagate one satellite. Hours 0 to 8: build the screening pipeline. Hours 8 to 18: multi-conjunction planning logic. Hours 18 to 24: support integration.

Agent Architect. Owns Nemotron integration, OpenClaw orchestration, the reasoning loop, tool definitions, prompt engineering, memory, NemoClaw wrapping. Day 0: verify OpenClaw works, get a Nemotron model loaded on hardware, prompt-iterate against mock tools. Hours 0 to 6: define and stub tools. Hours 6 to 14: build the reasoning loop. Hours 14 to 20: connect real tools, add NemoClaw if applicable. Hours 20 to 24: prompt tuning for demo.

3D Viz Lead. Owns Three.js Earth, satellite rendering (instanced meshes — do not create 1000 individual Meshes), conjunction highlights, trail rendering for highlighted objects, camera tweens to conjunction events, maneuver application animations. Day 0: Three.js boilerplate with Earth and fake satellite dots. Hours 0 to 6: real positions from Orbital Engine. Hours 6 to 14: conjunction highlights and camera work. Hours 14 to 20: maneuver application. Hours 20 to 24: polish.

UI and Integration Lead. Owns the React shell, Operator and Approver views, KPI strip, agent activity panel, plan approval cards, timeline strip, the data wiring across all four workstreams. Day 0: React + Tailwind + shadcn scaffold, layouts from sketches, mock data flowing. Hours 0 to 6: Operator View shell. Hours 6 to 14: Approver View. Hours 14 to 20: detail drill-down, polish. Hours 20 to 24: integration debugging.

Data contracts (lock these in hour 0): SatellitePosition has id, x, y, z, t (ECI coordinates). ConjunctionEvent has id, obj1, obj2, TCA, miss_distance, relative_velocity, Pc. AgentActivity has timestamp, type, content, related_conjunction_id. ManeuverPlan has plan_id, asset_id, conjunctions_resolved, burns, total_dv, alternatives. ApprovedPlan has plan_id, approver, timestamp.

9. The 24-Hour Plan

Hour 0: Kickoff. Confirm contracts. Confirm role boundaries. Set checkpoint times.

Hours 0 to 4: Data plumbing. Celestrak fetcher. SGP4 propagation working. Three.js Earth with fake satellites rendering. Mock data flowing through UI shell. Mock agent reasoning loop running against stub tools.

Hour 4: First standup. What is working, what is blocked.

Hours 4 to 10: Real conjunction screening producing real events. Real satellites rendering in 3D. Agent loop connected to real tools. Approver View skeleton working.

Hour 10: Mid-checkpoint. End-to-end integration test. Even if it is ugly, data should flow from Engine through Agent to UI.

Hours 10 to 18: Multi-conjunction planning logic. Maneuver simulation and application. Plan evaluation and recommendation generation. Camera choreography for conjunction events. Detail drill-down drawer.

Hour 16: Demo dress rehearsal. Run the full demo flow start to finish, even if rough.

Hours 16 to 20: NemoClaw wrapping if applicable. Visual polish. UI polish. Bug fixes.

Hour 20: Lock features. No new features after this point. Polish and bugfix only.

Hours 20 to 23: Polish and debugging. Make it pretty. Make it stable.

Hour 23: Final demo rehearsal.

Hour 24: Present.

10. Scope Cuts (Decided in Advance)

If we are behind at hour 12, we cut **NemoClaw**. We compete on the Edge Track only. No bonus track.

If we are behind at hour 14, we cut **multi-conjunction planning**. The agent handles one event at a time and recommends a single burn per event. Still demoable, still agentic, just less impressive.

If we are behind at hour 16, we cut **the Approver View** and demonstrate the agent's outputs in the Operator View directly. The chain-of-command narrative becomes a "future work" mention rather than a demoed feature.

If we are behind at hour 18, we **freeze the 3D viz** at whatever state it is in and focus 100% on making the agent reasoning loop work cleanly for one scripted scenario. A clean demo of one scenario beats a broken demo of five.

We do not cut: the agent itself, real TLE data, the screening pipeline, the scripted demo scenario. Those are the irreducible core.

11. The Demo (3 Minutes)

The story arc.

0:00 to 0:20. Hook. "Low Earth orbit has over 22,000 tracked objects and 100,000 collision-avoidance maneuvers a year. Most operators triage these by hand or with six-figure tools. We built an autonomous on-device agent that does this triage and runs entirely on the GX10."

0:20 to 1:00. Set the scene. Show the Operator View. Earth rotating. ~800 satellites visible. KPI strip showing live numbers. Brief explanation: "We are tracking the Starlink 550km shell, the ISS, and Fengyun-1C debris from the 2007 Chinese ASAT test. Real catalog. Real data. Updated from Celestrak. The agent runs on the GX10 next to me — no cloud."

1:00 to 2:00. The main event. Trigger the scripted scenario — Starlink-4521 vs Cosmos-2251 debris. Camera tweens to the conjunction. Agent activity stream lights up: pulling fresh TLE, checking space weather, computing refined Pc, evaluating maneuver plans. Show the agent generating two candidate plans and picking the better one. Recommendation appears.

2:00 to 2:40. Switch to Approver View. Show the recommendation card with reasoning. Approve the plan. Switch back to Operator View. Watch the satellite execute the burns. The conjunctions resolve. Markers turn green. The agent logs the outcome.

2:40 to 3:00. Close. "On-device. Multi-step agentic reasoning over real orbital data. Human in the loop. NemoClaw policy boundary means the orbital state never leaves the device. This is what edge AI for high-stakes domains looks like."

We pre-script the scenario so the demo is deterministic. We do not depend on a real-world conjunction happening at the right time.

12. Pre-Hackathon Homework (Before May 15)

Everyone: read this doc. Agree on the design. Raise objections now, not at hour 6.

Orbital Engine: install `sgp4` or `satellite.js`, parse a Celestrak TLE, propagate one satellite to "right now," print its lat/lon/altitude. ~30 minutes.

Agent Architect: verify OpenClaw exists and works. Find out which Nemotron variants fit on the GX10 (probably 8B to 70B quantized, NOT 340B). Read NemoClaw docs if they exist. Set up a local prompt iteration sandbox using mock tools and the Claude or OpenAI API — iterate the reasoning loop until it works against canned data. Several hours of prompt work that pays off massively at the hackathon.

3D Viz Lead: spin up a Three.js project with an Earth sphere, NASA Blue Marble texture, orbit camera, and 1000 randomly positioned dots animating around it. ~2 hours. This entire pipeline transfers directly to the hackathon.

UI Lead: Figma the Operator and Approver views based on the paper sketches. Lock the component library (Tailwind + shadcn recommended). Set up the React scaffold with mock data flowing through every panel. ~3 hours.

Everyone: decide who presents. The presenter rehearses the 3-minute script three times before May 15.

13. Open Questions / Risks

OpenClaw and NemoClaw. We have not verified these frameworks are mature enough to use. Day-0 verification is the single biggest risk. If they do not work, we fall back to a direct Nemotron integration without the orchestration framework.

Nemotron model size. The GX10 has 128GB unified memory. A 340B model does not fit even quantized. We are planning around 8B to 70B variants. Verify on day 0.

Confirm hardware access. Email community@nvidia.com or check the Discord — confirm whether GX10s are provided on-site, shared, or whether we develop on personal hardware and deploy at the end.

Space-Track CDM access. If we want real CDMs in the demo, registration approval is needed and can take days. Apply now even if uncertain.

Inference latency. If a single reasoning step on Nemotron takes 5+ seconds on the GX10, a full investigate loop is 30 to 60 seconds. May feel slow live. Mitigations: pre-warm, batch tool calls, show a thinking indicator.

14. Definition of Done

For the submission, Orbital is "working" when it can do all of the following end-to-end without manual intervention:

The screening engine detects a flagged conjunction within seconds of it appearing. The agent runs the full Investigate loop, streaming at least 5 distinct reasoning thoughts to the UI. It classifies the event correctly as dismiss, watch, or action based on refined Pc. For action-required events, it generates at least two candidate plans, evaluates each, and produces a recommendation with reasoning. The recommendation appears in the Approver View. A human approves the plan, the simulator applies it, the conjunction resolves in the 3D viz, and memory logs the full event chain.

If this works for one scripted scenario, we are demo-ready. If it works for three of our five scripted scenarios, we are hackathon-grade.

15. North Star

We are not curing cancer. We are not the first team to do conjunction analysis with AI — LeoLabs, Kayhan, and Slingshot already do it at industrial scale. What we are doing is showing

a directional shift: a problem currently solved by 50-person teams with six-figure budgets, approached by a 4-person team with an on-device LLM in 24 hours. We are a proof-of-concept for what on-device agent reasoning makes possible in high-stakes domains.

That is the project. Let us go build it.